



JS

JavaScript OOP

دليل تعليمي مختصر و منظم

٦ مارس ٢٠٢٦



جدول المحتويات

1. مقدمة في OOP — ما هي البرمجة الكائنية (OOP)؟
2. مقدمة في OOP — المفاهيم الأساسية: Object, Class, Property, Method.
3. مقدمة في OOP — مبادئ OOP: Abstraction, Encapsulation, Inheritance, Polymorphism.
4. مقدمة في OOP — لماذا نستخدم OOP في JavaScript؟
5. الكائنات في JavaScript — إنشاء الكائنات باستخدام Object Literals.
6. الكائنات في JavaScript — الوصول إلى الخصائص والتوابع وتعديلها.
7. الكائنات في JavaScript — إنشاء الكائنات باستخدام Factory Functions.
8. الكائنات في JavaScript — مشكلة تكرار التوابع.
9. فهم this في JavaScript — ما هي this وكيف يعمل السياق؟
10. فهم this في this في JavaScript — الدوال العادية وتوابع الكائن.
11. فهم this في JavaScript — تغيير السياق باستخدام call و apply و bind.
12. فهم this في this في JavaScript — Arrow Functions.
13. Prototype و Constructors — إنشاء الكائنات باستخدام Constructor Functions.
14. Prototype و Constructors — مفهوم Prototype في JavaScript.

- 15. Prototype — Prototype Chain g Constructors.
- 16. Prototype g Constructors — إضافة التوابع إلى Prototype لتحسين الأداء.
- 17. Classes في ES6 — مقدمة إلى class في JavaScript.
- 18. Classes في constructor — ES6 داخل class.
- 19. Classes في ES6 — تعريف الخصائص والتوابع.
- 20. Classes في Static Methods — Static Properties g ES6.
- 21. الوراثة (Inheritance) — الوراثة باستخدام extends.
- 22. الوراثة (Inheritance) — استخدام super().
- 23. الوراثة (Inheritance) — تجاوز التوابع (Method Overriding).
- 24. التغليف والخصائص الخاصة — مفهوم Encapsulation.
- 25. Abstraction g Polymorphism — محاكاة Interfaces في JavaScript.
- 26. Design Patterns الأساسية — Factory Pattern.
- 27. Design Patterns الأساسية — Singleton Pattern.
- 28. Design Patterns الأساسية — Observer Pattern.

مقدمة في OOP — ما هي البرمجة الكائنية (OOP)؟

ما هي البرمجة الكائنية (OOP)؟

البرمجة الكائنية التوجه (Object-Oriented Programming - OOP) هي نموذج برمجي يركز على تنظيم الكود حول الكائنات (Objects) بدلاً من الوظائف والإجراءات. فكل شيء في العالم من حولك؛ إنه مليء بالكائنات: سيارات، هواتف، أشخاص. كل كائن له خصائص (مثل اللون، الحجم) وسلوكيات (مثل القيادة، الاتصال).

المفهوم الرئيسي

في OOP، نحاول محاكاة هذا العالم الحقيقي في تصميم برامجننا. بدلاً من كتابة سلسلة من التعليمات خطوة بخطوة، نقوم بإنشاء 'كائنات' رقمية تحتوي على البيانات (الخصائص) والوظائف (السلوكيات) التي تعمل على هذه البيانات. هذا يجعل الكود أكثر تنظيماً، وقابلية للفهم، وسهل الإدارة.

مثال برمجي

```
const car = {
  brand: "Toyota",
  model: "Camry",
  year: 2020,
  start: function() {
    console.log("The car has started.");
  },
  stop: function() {
    console.log("The car has stopped.");
  }
};

console.log(car.brand); // الوصول إلى خاصية
car.start();           // استدعاء سلوك (منهج)
```

شرح المثال

في هذا المثال، أنشأنا كائن `car` يمثل سيارة. هذا الكائن لديه خصائص مثل `brand` و `model` و `year`، ولديه أيضًا سلوكيات (وظائف) مثل `start` و `stop`. هذا يعكس كيف يمكن للكائن في OOP أن يجمع بين البيانات والوظائف المتعلقة بها في وحدة واحدة.

- تساعد OOP في تنظيم الكود بطريقة تحاكي العالم الحقيقي.
- تركز على الكائنات كوحدات أساسية للبناء.
- يجمع كل كائن بين البيانات (الخصائص) والوظائف (السلوكيات) الخاصة به.

مقدمة في OOP — المفاهيم الأساسية: Object, Class, Property, Method

المفاهيم الأساسية: Object, Class, Property, Method

للتعمق في البرمجة الكائنية، من الضروري فهم أربعة مفاهيم أساسية تشكل لبنات البناء الرئيسية: الكائن (Object)، الفئة (Class)، الخاصية (Property)، والمنهج (Method).

المفاهيم الرئيسية

- **الفئة (Class):** هي بمثابة مخطط أو قالب لإنشاء الكائنات. تحدد الخصائص والوظائف المشتركة التي ستمتلكها جميع الكائنات التي يتم إنشاؤها من هذه الفئة. فكر فيها كرسومات هندسية لسيارة.
- **الكائن (Object):** هو مثيل ملموس للفئة. هو كيان حقيقي يتم إنشاؤه من الفئة. بناءً على رسومات السيارة (الفئة)، يمكنك بناء سيارة حقيقية (الكائن) واحدة أو أكثر.
- **الخاصية (Property):** هي سمة أو صفة تميز الكائن. تصف حالة الكائن. مثل لون السيارة، أو نوعها، أو سنة صنعها.
- **المنهج (Method):** هو وظيفة أو سلوك يمكن للكائن أن يؤديه. يصف الإجراءات التي يمكن للكائن القيام بها. مثل قيادة السيارة، أو إيقافها، أو تشغيل محركها.

مثال برمجي

```
class Dog {  
  constructor(name, breed) {  
    this.name = name; // خاصية  
    this.breed = breed; // خاصية  
  }  
  
  bark() { // منهج  
    console.log(`${this.name} says Woof!`);  
  }  
}
```

```

describe() { // منهج آخر
  console.log(`${this.name} is a ${this.breed}.`);
}
}

const myDog = new Dog("Buddy", "Golden Retriever"); // إنشاء كائن
const anotherDog = new Dog("Lucy", "Poodle"); // إنشاء كائن آخر

myDog.bark(); // استدعاء منهج على الكائن
anotherDog.describe(); // استدعاء منهج على الكائن

```

شرح المثال

في هذا المثال، `Dog` هي **الفئة** التي تعرف خصائص كل كلب (`name` و `breed`) وسلوكياته (`bark()` و `describe()`). `name` و `breed` هما **خاصيتان**. `bark()` و `describe()` هما **منهجان**. عند استخدام `new Dog(...)`، نقوم بإنشاء **كائنات** جديدة (`myDog` و `anotherDog`) من الفئة `Dog`، وكل كائن يمتلك نسخته الخاصة من هذه الخصائص والمنهجيات.

- الفئة هي مخطط، الكائن هو تجسيد لهذا المخطط.
- الخاصية هي سمة أو بيان للكائن.
- المنهج هو وظيفة أو سلوك يمكن للكائن القيام به.

مقدمة في OOP — مبادئ، Abstraction، Encapsulation، Inheritance، Polymorphism.

مبادئ، Abstraction، Encapsulation، Inheritance، Polymorphism.

تقوم البرمجة الكائنية على أربعة مبادئ أساسية تجعلها قوية ومرنة: التجريد (Abstraction)، التغليف (Encapsulation)، الوراثة (Inheritance)، وتعدد الأشكال (Polymorphism). هذه المبادئ تساعد في بناء أنظمة برمجية منظمة، قابلة للصيانة، وقابلة للتوسع.

المفاهيم الرئيسية

- **التجريد (Abstraction):** التركيز على ما يهم وإخفاء التفاصيل غير الضرورية. يظهر الكائن واجهة بسيطة للمستخدم بينما يتم التعامل مع التعقيدات الداخلية خلف الكواليس. فكر في قيادة السيارة: أنت تستخدم عجلة القيادة والدواسات دون الحاجة لمعرفة تفاصيل عمل المحرك.
- **التغليف (Encapsulation):** تجميع البيانات (الخصائص) والوظائف (المنهجيات) التي تعمل عليها في وحدة واحدة (الكائن)، وحماية البيانات الداخلية من الوصول أو التعديل الخارجي المباشر. هذا يحافظ على سلامة البيانات ويقلل من الآثار الجانبية غير المتوقعة.
- **الوراثة (Inheritance):** تسمح لفئة جديدة (الفئة الفرعية) باكتساب الخصائص والمنهجيات من فئة موجودة (الفئة الأساسية). هذا يعزز قابلية إعادة استخدام الكود ويساعد على إنشاء تسلسل هرمي للكائنات.
- **تعدد الأشكال (Polymorphism):** تعني 'أشكال متعددة'. هي القدرة على معالجة الكائنات من فئات مختلفة بطريقة موحدة إذا كانت تشترك في نفس الواجهة أو الفئة الأساسية. يسمح للكائنات المختلفة بالاستجابة لنفس الرسالة بطرق فريدة خاصة بها.

مثال برمجي

```
// مثال على الوراثة
class Animal {
    constructor(name) {
```

```

        this.name = name;
    }

    speak() {
        console.log(`${this.name} makes a sound.`);
    }
}

class Cat extends Animal { // Cat ترث من Animal
    constructor(name) {
        super(name); // استدعاء مُنشئ الفئة الأساسية
    }

    speak() { // طريقة الخاصة speak تطبق Cat: تعدد الأشكال
        console.log(`${this.name} meows.`);
    }
}

class Dog extends Animal { // Dog ترث من Animal
    constructor(name) {
        super(name);
    }

    speak() { // طريقة الخاصة speak تطبق Dog: تعدد الأشكال
        console.log(`${this.name} barks.`);
    }
}

const myCat = new Cat("Whiskers");
const myDog = new Dog("Buddy");

myCat.speak(); // Whiskers meows.
myDog.speak(); // Buddy barks.

```

شرح المثال

في هذا المثال، لدينا فئة أساسية `Animal` وفئتان فرعيتان `Cat` و `Dog` ترثان منها. هذا يوضح **الوراثة**، حيث ترث `Cat` و `Dog` خاصية `name` ومنهج `speak()` من `Animal`. كما يوضح المثال **تعدد الأشكال**، حيث تقوم كل من `Cat` و `Dog` بتنفيذ منهج `speak()` بطريقة الخاصة، على الرغم من أنهما تستدعيان نفس اسم المنهج. يتم هنا التجريد من خلال التركيز على السلوك العام (الصوت) والتغليف من خلال احتواء الاسم وسلوك التحديث داخل الكائن.

- التجريد يُبسط الواجهة ويُخفي التفاصيل المعقدة.
- التغليف يحمي البيانات ويجمع الخصائص والمنهجيات.
- الوراثة تسمح بإعادة استخدام الكود وإنشاء تسلسلات هرمية.
- تعدد الأشكال يُمكن الكائنات المختلفة من الاستجابة بطرق فريدة لنفس الرسالة.

مقدمة في OOP — لماذا نستخدم OOP في JavaScript؟

لماذا نستخدم OOP في JavaScript؟

تتمتع JavaScript بمرونة كبيرة في أنماط البرمجة، ولكن استخدام البرمجة الكائنية التوجه (OOP) فيها يجلب العديد من الفوائد التي تجعلها خيارًا شائعًا لتطوير تطبيقات معقدة وقابلة للصيانة.

الفوائد الرئيسية

على الرغم من أن JavaScript لغة قائمة على النماذج الأولية (prototypal) وليست كلاسكية (class-based) تقليدية، إلا أنها توفر دعمًا قويًا لمفاهيم OOP من خلال بناء الجملة الحديث (ES6 Classes). استخدام OOP في JavaScript يساعد في:

- **تحسين تنظيم الكود:** تجميع البيانات والوظائف ذات الصلة في كائنات وفئات يمنح بنية واضحة ومنظمة، مما يسهل فهم الكود وتتبع الأخطاء.
- **إعادة استخدام الكود (Reusability):** باستخدام الوراثة والفئات، يمكنك بناء كود يمكن استخدامه في أجزاء مختلفة من التطبيق أو حتى في مشاريع أخرى، مما يوفر الوقت والجهد.
- **سهولة الصيانة والتوسع (Maintainability & Scalability):** الكود المنظم في كائنات يسهل تعديله وتوسيعه. عند الحاجة لتغيير سلوك معين، يمكنك غالبًا تحديث الفئة أو الكائن المعني دون التأثير على أجزاء أخرى من النظام.
- **التعامل مع التعقيد:** في المشاريع الكبيرة، يمكن أن يصبح الكود متشابكًا ومعقدًا. توفر OOP طريقة لتقسيم المشكلة إلى أجزاء أصغر وأكثر قابلية للإدارة (الكائنات)، مما يقلل من التعقيد الكلي.

مثال برمجي

```
class UserProfile {  
  constructor(username, email) {  
    this.username = username;  
    this.email = email;  
  }  
}
```

```

    }

    displayInfo() {
        console.log(`Username: ${this.username}, Email: ${this.email}`);
    }

    updateEmail(newEmail) {
        this.email = newEmail;
        console.log(`Email updated to: ${this.email}`);
    }
}

const user1 = new UserProfile("ahmed_dev", "ahmed@example.com");
user1.displayInfo(); // يعرض معلومات المستخدم
user1.updateEmail("ahmed.new@example.com"); // يحدد البريد الإلكتروني
user1.displayInfo();

```

شرح المثال

يوضح هذا المثال كيف يمكننا استخدام فئة `UserProfile` لتمثيل مستخدم في تطبيقنا. كل كائن `UserProfile` يجمع بين البيانات (`username` , `email`) والوظائف (`displayInfo` , `updateEmail`) المتعلقة به. هذا التنظيم يجعل من السهل إنشاء مستخدمين جدد، والوصول إلى معلوماتهم، وتعديلها بطريقة متسقة ومنظمة، مما يعزز جميع فوائد OOP المذكورة.

- تساهم OOP في جعل كود JavaScript أكثر تنظيمًا ووضوحًا.
- توفر آليات قوية لإعادة استخدام الكود وتسهيل صيانه.
- تساعد في بناء تطبيقات JavaScript كبيرة ومعقدة بكفاءة.

الكائنات في JavaScript — إنشاء الكائنات باستخدام Object Literals.

إنشاء الكائنات باستخدام Object Literals.

تُعد الكائنات (Objects) من اللبنات الأساسية في JavaScript، وهي تسمح لنا بتجميع البيانات والوظائف ذات الصلة في وحدة واحدة. أسهل طريقة لإنشاء كائن واحد هي باستخدام Object Literals، والتي تُعرف أيضًا باسم Object Initializers.

مفهوم أساسي: Object Literals

Object Literal هي طريقة بسيطة ومباشرة لإنشاء كائن واحد. يتم تعريفها باستخدام الأقواس المتعرجة `{ }`، وتحتوي على أزواج من **المفتاح (Key)** و**القيمة (Value)** مفصولة بنقطتين رأسيتين `:`. المفتاح هو اسم الخاصية، والقيمة يمكن أن تكون أي نوع من البيانات، بما في ذلك وظائف (توابع).

مثال برمجي

```
const car = {
  make: 'Toyota',
  model: 'Corolla',
  year: 2023,
  isElectric: false,
  start: function() {
    console.log('السيارة تعمل الآن');
  },
  stop: () => {
    console.log('تم إيقاف السيارة');
  }
};

console.log(car.model); // Corolla
car.start(); // السيارة تعمل الآن
```

شرح المثال

في هذا المثال، قمنا بإنشاء كائن `car` باستخدام Object Literal. يحتوي الكائن على خصائص مثل `make` و `model` و `year` و `isElectric`، بالإضافة إلى تابعين (وظيفتين) هما `start` و `stop`. يمكننا الوصول إلى الخصائص واستدعاء التوابع باستخدام تدوين النقطة (Dot Notation).

- Object Literal هي أبسط طريقة لإنشاء كائن واحد.
- تتكون من أزواج `key: value` محاطة بـ `{ }`.
- يمكن أن تحتوي على خصائص (بيانات) وتوابع (وظائف).

الكائنات في JavaScript — الوصول إلى الخصائص والتوابع وتعديلها.

الوصول إلى الخصائص والتوابع وتعديلها.

بعد إنشاء الكائن، نحتاج إلى القدرة على قراءة قيمه وتغييرها، وكذلك استدعاء وظائفه. توفر JavaScript طرقًا مرنة للوصول إلى خصائص الكائن وتعديلها واستدعاء توابعه.

مفهوم أساسي: الوصول والتعديل

يمكن الوصول إلى خصائص الكائن باستخدام طريقتين رئيسيتين: **Dot Notation** (تدوين النقطة) و **Bracket Notation** (تدوين الأقواس المربعة). لتعديل قيمة خاصية موجودة، نقوم بإعادة تعيينها باستخدام علامة التساوي `=`. لاستدعاء تابع، نستخدم اسمه متبوعًا بالأقواس `()`.

مثال برمجي

```
const user = {
  firstName: 'أحمد',
  lastName: 'علي',
  age: 28,
  email: 'ahmad.ali@example.com',
  greet: function() {
    console.log(`مرحباً، أنا ${this.firstName} ${this.lastName}.`);
  }
};

// الوصول إلى الخصائص
console.log(user.firstName); // أحمد (Dot Notation)
console.log(user['email']); // ahmad.ali@example.com (Bracket Notation)

// تعديل خاصية
user.age = 29;
console.log(user.age); // 29
```



```
// إضافة خاصية جديدة
user.city = 'الرياض';
console.log(user.city);           // الرياض

// استدعاء تابع
user.greet();                     // مرحباً، أنا أحمد علي
```

شرح المثال

يوضح المثال كيف يمكننا الوصول إلى `firstName` باستخدام تدوين النقطة وإلى `email` باستخدام تدوين الأقواس. قمنا بتعديل قيمة الخاصية `age` من 28 إلى 29، وأضفنا خاصية جديدة `city`. أخيرًا، استدعينا التابع `greet()` الذي يقوم بطباعة رسالة ترحيب باستخدام خصائص الكائن.

- يمكن الوصول للخصائص باستخدام تدوين النقطة `object.property` أو تدوين الأقواس `object['property']`.
- تُعدّل قيم الخصائص بإعادة تعيينها: `object.property = newValue`.
- تُستدعى التوابيع باستخدام اسمها متبوعًا بالأقواس `()`.

الكائنات في JavaScript — إنشاء الكائنات باستخدام Factory Functions.

إنشاء الكائنات باستخدام Factory Functions.

عندما نحتاج إلى إنشاء العديد من الكائنات التي تشترك في نفس البنية والوظائف، يصبح استخدام Object Literals بشكل متكرر أمرًا غير عملي ويؤدي إلى تكرار الكود. هنا يأتي دور Factory Functions كحل بسيط وفعال.

مفهوم أساسي: Factory Functions

Factory Function هي ببساطة دالة عادية تُعيد كائنًا جديدًا. تسمح لنا بإنشاء كائنات متعددة ذات بنية متشابهة بسهولة ومرونة، حيث يمكن تمرير وسيطات (arguments) إلى الدالة لتخصيص خصائص الكائن الجديد عند إنشائه. لا يتطلب الأمر استخدام الكلمة المفتاحية `new` مع Factory Functions.

مثال برمجي

```
function createPerson(name, age) {
  return {
    name: name,
    age: age,
    greet: function() {
      console.log(`${this.age} سنة و عمري ${this.name} مرحباً، أنا`);
    },
    celebrateBirthday: function() {
      this.age++;
      console.log(`${this.age} لقد أصبحت! ${this.name} عيد ميلاد سعيد يا`);
    }
  };
}

// إنشاء كائنين باستخدام ال Factory Function
const person1 = createPerson('سارة', 30);
const person2 = createPerson('خالد', 25);
```

```
person1.greet(); // مرحباً، أنا سارة وعمري 30 سنة
person2.celebrateBirthday(); // لقد أصبحت 26 سنة الآن
person1.greet(); // مرحباً، أنا سارة وعمري 30 سنة
```

شرح المثال

في هذا المثال، أنشأنا `createPerson` ك Factory Function. تأخذ هذه الدالة اسماً وعمراً، وتُعيد كائن شخص يحتوي على هذه الخصائص بالإضافة إلى تابعين هما `greet` و `celebrateBirthday`. لاحظ كيف يمكننا إنشاء كائنات `person1` و `person2` بشكل مستقل، كل بخصائصه الخاصة، وجميعها تستخدم نفس بنية الكائن ووظائفه.

- Factory function هي دالة عادية تُعيد كائناً جديداً.
- تُستخدم لإنشاء كائنات متعددة ذات بنية متشابهة بطريقة منظمة.
- تُمكن من تخصيص خصائص الكائن عند إنشائه عن طريق تمرير وسيطات.

الكائنات في JavaScript — مشكلة تكرار التوابع.

مشكلة تكرار التوابع.

على الرغم من أن Factory Functions توفر طريقة رائعة لإنشاء كائنات متعددة، إلا أنها تأتي مع مشكلة خفية تتعلق بكيفية تخزين التوابع (Methods) في الذاكرة، خاصة عندما نتعامل مع عدد كبير من الكائنات.

مفهوم أساسي: تكرار التوابع في الذاكرة

عندما نستخدم Factory Function لإنشاء كائنات، فإن كل كائن جديد يتم إنشاؤه يحصل على نسخة **خاصة به** من كل تابع معرف داخل الدالة. هذا يعني أنه إذا كان لدينا 100 كائن، وكل كائن يحتوي على تابع `greet`، فإن هناك 100 نسخة مختلفة من الدالة `greet` مخزنة في الذاكرة. هذا التكرار غير ضروري ويمكن أن يؤدي إلى استهلاك غير فعال للذاكرة، خاصة إذا كانت التوابع معقدة أو كان عدد الكائنات كبيرًا جدًا.

مثال برمجي

```
function createAnimal(name, species) {
  return {
    name: name,
    species: species,
    makeSound: function() {
      console.log(`${this.name} (${this.species}) يصدر صوتاً`);
    },
    eat: function() {
      console.log(`${this.name} يتناول طعامه`);
    }
  };
};

const dog = createAnimal('كلب', 'باستر');
const cat = createAnimal('قطعة', 'مياو');
const bird = createAnimal('طائر', 'زقزوق');
```

```
console.log(dog.makeSound === cat.makeSound); // الناتج : false
console.log(cat.eat === bird.eat);           // الناتج : false
```

// كل كائن لديه نسخة فريدة من التوابع الخاصة به في الذاكرة

شرح المثال

في المثال أعلاه، أنشأنا Factory Function تُسمى `createAnimal`. قمنا بإنشاء ثلاثة كائنات (`dog`, `cat`, `bird`) باستخدام هذه الدالة. عندما نقارن تابعي `makeSound` للكائنين `dog` و `cat`، نجد أن النتيجة هي `false`. هذا يؤكد أن كل كائن لديه نسخة مختلفة تمامًا من التابع `makeSound` في الذاكرة، على الرغم من أن وظيفتها متطابقة. هذه هي مشكلة تكرار التوابع التي تجعلنا نبحث عن حلول أكثر كفاءة في إدارة الذاكرة.

- كل كائن يتم إنشاؤه بواسطة Factory Function يحصل على نسخة خاصة به من التوابع.
- هذا يؤدي إلى استهلاك زائد للذاكرة عند إنشاء العديد من الكائنات.
- تكرار التوابع يقلل من كفاءة إدارة الموارد في التطبيقات الكبيرة.

فهم this في JavaScript — ما هي this وكيف يعمل السياق؟

ما هي this وكيف يعمل السياق؟

كلمة المفتاح **this** في JavaScript هي واحدة من أكثر المفاهيم التي تسبب حيرة للمبتدئين وحتى للمطورين ذوي الخبرة. قيمتها ليست ثابتة، بل تتغير ديناميكيًا بناءً على كيفية استدعاء الدالة أو السياق الذي يتم استخدامها فيه.

المفهوم الأساسي: السياق العالمي (Global Context)

عندما تُستخدم **this** خارج أي دالة أو كائن في السياق العام (أي في أعلى مستوى من ملف JavaScript)، فإنها تشير إلى الكائن العام. في المتصفحات، يكون هذا الكائن هو `window`، وفي Node.js، يكون `global`.

مثال على الكود

```
// في السياق العام (خارج أي دالة)
console.log(this);

// داخل دالة عادية معرفة في السياق العام
function showGlobalThis() {
  console.log(this);
}
showGlobalThis();
```

الشرح

في المثال أعلاه، كلا استدعاء `console.log(this)` سيخرجان الكائن العام (مثل `Window` في المتصفح). هذا لأننا في السياق الأوسع للتنفيذ. عند تعريف دالة عادية في السياق العام واستدائها

بشكل مباشر، فإنها أيضًا تشير إلى الكائن العام تلقائيًا.

- **this** كلمة مفتاحية خاصة بتغير قيمتها.
- قيمتها تتحدد بناءً على كيفية استدعاء الدالة أو موقعها.
- في السياق العالمي، تشير **this** دائمًا إلى الكائن العام (`global` أو `window`).

فهم this في JavaScript — الدوال العادية وتوابع الكائن.

this في الدوال العادية وتوابع الكائن.

فهم كيفية تصرف **this** في الدوال العادية مقابل توابع الكائنات (methods) أمر بالغ الأهمية. هذا هو المكان الذي يبدأ فيه سلوك **this** بالتباين بشكل واضح، وهو مفتاح لفهم البرمجة كائنية التوجه في JavaScript.

المفهوم الأساسي: الدوال كدوال عادية مقابل كطرق للكائن

عند استدعاء دالة كدالة مستقلة (regular function call)، فإن قيمة **this** غالبًا ما تشير إلى الكائن العام. أما عند استدعاء دالة كطريقة لكائن معين (method call)، فإن **this** تشير إلى الكائن الذي تم استدعاء الدالة عليه.

مثال على الكود

```
// دالة عادية
function greet() {
  console.log('مرحباً، أنا', this.name || 'مجهول');
}
greet(); // استدعاء عادي

// كائن مع تابع (method)
const person = {
  name: 'أحمد',
  sayHello: function() {
    console.log('مرحباً، أنا', this.name);
  }
};
person.sayHello(); // استدعاء كطريقة للكائن

// دالة معارة لكائن آخر
const anotherPerson = {
```



```
name: 'فاطمة'
};
anotherPerson.greetMethod = greet; // إعادة استخدام الدالة
anotherPerson.greetMethod();
```

الشرح

في استدعاء `greet()` الأول، لا يوجد سياق محدد، لذا تشير `this` إلى الكائن العام، والذي لا يحتوي على `name`، فيطبع 'مجهول'. عندما يتم استدعاء `person.sayHello()`، فإن `this` تشير إلى الكائن `person`، ولذلك تطبع 'أحمد'. وبالمثل، عند تعيين `greet` كطريقة لـ `anotherPerson`، فإن استدعاء `anotherPerson.greetMethod()` يجعل `this` تشير إلى `anotherPerson`، فتطبع 'فاطمة'.

- في الدوال العادية، `this` تشير إلى الكائن العام (أو `undefined` في الوضع الصارم).
- عند استدعاء دالة كطريقة لكائن، تشير `this` إلى ذلك الكائن الذي تم استدعاء الدالة عليه.
- قيمة `this` تتحدد ديناميكيًا وقت استدعاء الدالة، وليس وقت تعريفها.

فهم this في JavaScript — تغيير السياق باستخدام call و apply و bind.

تغيير السياق باستخدام call و apply و bind.

في بعض الأحيان، نحتاج إلى التحكم بشكل صريح في قيمة **this** داخل دالة، خاصةً عندما تكون الدالة قد قُطعت عن الكائن الأصلي أو عند التعامل مع دوال رد الاتصال. هنا تأتي أهمية دوال `call` و `apply` و `bind`.

المفهوم الأساسي: الأدوات call و apply و bind

هذه الدوال هي توابع لـ `Function.prototype` وتسمح لك باستدعاء دالة مع تحديد كائن معين ليكون سياق **this** الخاص بها. الفرق الرئيسي يكمن في كيفية تمرير الوسائط وكيفية معالجة الدالة.

مثال على الكود

```
function introduce(job, city) {  
  console.log(`أنا ${this.name}، أعمل كـ ${job} في ${city}.`);  
}  
  
const person1 = { name: 'سارة' };  
const person2 = { name: 'علي' };  
  
// تمرير الوسائط بشكل فردي باستخدام call  
introduce.call(person1, 'مهندسة', 'دبي');  
  
// تمرير الوسائط كمصفوفة باستخدام apply  
introduce.apply(person2, ['طبيب', 'الرياض']);  
  
// محددة بشكل دائم this لإنشاء دالة جديدة بقيمة bind استخدام  
const introduceSarah = introduce.bind(person1, 'مديرة');  
introduceSarah('القاهرة'); // هنا يتم تمرير 'القاهرة' فقط  
  
const introduceAliInJeddah = introduce.bind(person2, 'مدرس', 'جدة');
```

```
introduceAliInJeddah(); // لا حاجة لتمرير وسائط بعد الآن
```

الشرح

كل من `call` و `apply` تقومان بتشغيل الدالة `introduce` فورًا، مع تعيين `person1` و `person2` على التوالي كقيمة `this`. الفرق يكمن في طريقة تمرير الوسائط: `call` تأخذها واحدة تلو الأخرى، بينما `apply` تأخذها كمصفوفة. أما `bind`، فلا تشغل الدالة مباشرة، بل ترجع دالة جديدة بقيمة `this` محددة بشكل دائم، ويمكنك استدعاء هذه الدالة الجديدة لاحقًا.

- `call`: تستدعي الدالة فورًا مع تحديد `this` وتمرير الوسائط بشكل فردي.
- `apply`: تستدعي الدالة فورًا مع تحديد `this` وتمرير الوسائط كمصفوفة.
- `bind`: تُرجع دالة جديدة بقيمة `this` مرتبطة بشكل دائم، ولا تشغل الدالة مباشرة.

فهم this في JavaScript مع Arrow Functions.

this مع Arrow Functions.

لقد قدمت دوال الأسهم (Arrow Functions) تغييرًا مهمًا في كيفية عمل **this** في JavaScript. على عكس الدوال العادية، لا تمتلك دوال الأسهم سياق **this** الخاص بها، مما يحل العديد من المشكلات الشائعة التي يواجهها المطورون.

المفهوم الأساسي: this Lexical Scoping

دوام الأسهم ترث قيمة **this** من السياق المحيط الذي تم تعريفها فيه (lexical context)، أي من الدالة الأبوية أو الكائن الذي يحتويها. هذا يعني أن قيمة **this** لدالة السهم لا تتغير أبدًا عند استدعائها، بغض النظر عن كيفية استدعائها.

مثال على الكود

```
const user = {
  name: 'ليلى',
  // تابع يستخدم دالة عادية كدالة رد اتصال
  greetOldWay: function() {
    console.log(`مرحباً من ${this.name}!`); // user هذا يشير إلى
    setTimeout(function() {
      console.log(`المتأخر: ${this.name || 'مجهول'}`); // this العام
    }, 1000);
  },
  // تابع يستخدم دالة سهم كدالة رد اتصال
  greetArrowWay: function() {
    console.log(`مرحباً من ${this.name}!`); // user هذا يشير إلى
    setTimeout(() => {
      console.log(`المتأخر (سهم): ${this.name}`); // this هنا يرث سياق
    }, 1000);
  }
};
```

```
user.greetOldWay();  
user.greetArrowWay();
```

الشرح

في `greetOldWay` ، دالة `setTimeout` تستخدم دالة عادية، وبالتالي تفقد سياق `this` الخاص بـ `user` وتعود للإشارة إلى الكائن العام، مما ينتج عنه 'مجهول'. في المقابل، `greetArrowWay` تستخدم دالة سهم لـ `setTimeout` ، مما يسمح لها بالاحتفاظ بسياق `this` الخاص بـ `user` (من الدالة نفسها) وتطبع 'ليلى' بشكل صحيح. هذا يجعل دوال الأسهم مثالية للاستخدام في دوال رد الاتصال داخل الكائنات.

- دوام الأسهم لا تملك سياق `this` الخاص بها.
- إنها ترث قيمة `this` من السياق المحيط بها الذي تم تعريفها فيه.
- هذا السلوك يجعلها مثالية للحفاظ على سياق `this` الصحيح في دوال رد الاتصال.

Prototype و Constructors — إنشاء الكائنات باستخدام Constructor Functions.

إنشاء الكائنات باستخدام Constructor Functions.

في عالم البرمجة الكائنية، غالبًا ما نحتاج إلى إنشاء العديد من الكائنات التي تشترك في نفس الخصائص والتوابع. بدلاً من كتابة كل كائن يدويًا، يمكننا استخدام Constructor Functions كقوالب.

المفهوم الأساسي: Constructor Functions

Constructor Function هي دالة JavaScript عادية ولكن يتم استدعاؤها باستخدام الكلمة المفتاحية `new`. وظيفتها الرئيسية هي بناء وتهيئة كائنات جديدة. عادةً ما تبدأ أسماء Constructor Functions بحرف كبير لتمييزها عن الدوال العادية.

مثال برمجي

```
function Person(name, age) {  
  this.name = name;  
  this.age = age;  
  this.greet = function() {  
    return `سنة ${this.age} وعمري ${this.name} مرحباً، اسمي`;  
  };  
}  
  
const person1 = new Person('علي', 30);  
const person2 = new Person('سارة', 25);  
  
console.log(person1.greet());  
console.log(person2.name);
```

الشرح

في هذا المثال، `Person` هي Constructor Function. عندما نستدعيها بـ `new Person (...)`، تقوم JavaScript بإنشاء كائن فارغ جديد، وتعيين `this` ليشير إلى هذا الكائن داخل الدالة، ثم تنفذ التعليمات داخل الدالة لتهيئة الكائن بالخصائص (`age`، `name`) والتابع (`greet`). في النهاية، يتم إرجاع الكائن الجديد ضمنيًا.

- Constructor Functions هي قوالب لإنشاء كائنات متشابهة.
- يتم استدعاؤها دائمًا باستخدام الكلمة المفتاحية `new`.
- `this` داخل Constructor Function يشير إلى الكائن الجديد الذي يتم بناؤه.

Prototype و Constructors — مفهوم

في JavaScript.

مفهوم Prototype في JavaScript.

بعد تعلم كيفية إنشاء الكائنات باستخدام Constructor Functions، من المهم فهم كيف تشارك هذه الكائنات التوابع والخصائص بطريقة فعالة وموفرة للذاكرة. هنا يأتي دور الـ Prototype.

المفهوم الأساسي: Prototype

في JavaScript، كل دالة لديها خاصية تسمى `prototype`، وهي كائن بحد ذاته. عندما تقوم بإنشاء كائن جديد باستخدام Constructor Function، فإن هذا الكائن الجديد يحصل على رابط إلى الكائن الخاص بالدالة البانية التي أنشأته. هذا يعني أن أي خصائص أو توابع تضيفها إلى `ConstructorFunction.prototype` ستكون متاحة لجميع الكائنات التي يتم إنشاؤها من هذه الدالة.

مثال برمجي

```
function Car(make, model) {  
  this.make = make;  
  this.model = model;  
}  
  
// إضافة تابع إلى Prototype  
Car.prototype.startEngine = function() {  
  return `يعمل ${this.make} ${this.model} محرك`;  
};  
  
const car1 = new Car('كامري', 'تويوتا');  
const car2 = new Car('سيفيك', 'هوندا');  
  
console.log(car1.startEngine());  
console.log(car2.startEngine());
```



```
console.log(car1.hasOwnProperty('startEngine')); // false
console.log(car1.__proto__.hasOwnProperty('startEngine')); // true
```

الشرح

في هذا المثال، تابع `startEngine` تمت إضافته إلى `Car.prototype` وليس مباشرة داخل Constructor Function. هذا يعني أن كلاً من `car1` و `car2` يمكنهما استدعاء `startEngine()` ، لكن نسخة واحدة فقط من هذا التابع موجودة في الذاكرة ويتم مشاركتها بينهما. هذا يوفر الذاكرة بشكل كبير مقارنةً بإنشاء نسخة جديدة من التابع لكل كائن.

- كل دالة في JavaScript لها خاصية `prototype` (وهي كائن).
- الكائنات التي يتم إنشاؤها بواسطة Constructor ترث من الـ `prototype` الخاص بالـ `Constructor`.
- يُستخدم الـ `prototype` لمشاركة التوابع والخصائص بين الكائنات لتقليل استهلاك الذاكرة.

Prototype — Prototype Chain g Constructors

Prototype Chain

بينما فهمنا أن الكائنات ترث من الـ prototype الخاص بها، كيف تعمل هذه الآلية عندما تكون هناك طبقات متعددة من الوراثة؟ هنا يظهر مفهوم Prototype Chain.

المفهوم الأساسي: Prototype Chain

الـ Prototype Chain (سلسلة النماذج الأولية) هي آلية في JavaScript تحدد كيفية بحث المحرك عن الخصائص والتوابع. عندما تحاول الوصول إلى خاصية أو تابع على كائن ما، فإن JavaScript تبدأ بالبحث في الكائن نفسه. إذا لم يتم العثور عليها، فإنها تنتقل تلقائيًا للبحث في الـ prototype الخاص بهذا الكائن (المشار إليه داخليًا بواسطة `__proto__` أو `Object.getPrototypeOf()`). إذا لم تجدها هناك أيضًا، فإنها تستمر في الصعود في السلسلة إلى الـ prototype الخاص بالـ prototype التالي، وهكذا، حتى تصل إلى `null` (الذي يعني نهاية السلسلة).

مثال برمجي

```
function Animal(name) {
  this.name = name;
}

Animal.prototype.speak = function() {
  return `يصدر صوتاً ${this.name}`;
};

function Dog(name, breed) {
  Animal.call(this, name); // الأب Constructor استدعاء
  this.breed = breed;
}

// Animal الخاص بـ Prototype بـ Dog الخاص بـ Prototype ربط
Dog.prototype = Object.create(Animal.prototype);
Dog.prototype.constructor = Dog; // الـ constructor تصحيح مرجع الـ
```

```

Dog.prototype.bark = function() {
  return `${this.name} !ينبح`;
};

const myDog = new Dog('جيرمان شيبيرد', 'كلبي');

console.log(myDog.bark()); // موجود في Dog.prototype
console.log(myDog.speak()); // موجود في Animal.prototype (عبر السلسلة)
console.log(myDog.toString()); // موجود في Object.prototype (على في السلسلة)

```

الشرح

في هذا المثال، `myDog` هو كائن من نوع `Dog`. عندما نستدعي `myDog.bark()`، تجد JavaScript التابع مباشرة في `Dog.prototype`. عندما نستدعي `myDog.speak()`، لا تجده في `myDog` ولا في `Dog.prototype`، فتصعد السلسلة لتجده في `Animal.prototype`. وعندما نستدعي `myDog.toString()`، لا تجده في أي من الـ prototypes السابقة، فتواصل الصعود لتجده في `Object.prototype`، وهو أعلى كائن في السلسلة قبل الوصول إلى `null`.

- الـ Prototype Chain هو المسار الذي تتبعه JavaScript للبحث عن الخصائص والتوابع.
- تبدأ البحث في الكائن نفسه، ثم تنتقل إلى الـ prototype الخاص به، وهكذا.
- تنتهي السلسلة عند `null`، بعد المرور بـ `Object.prototype`.

Prototype و Constructors — إضافة التوابع إلى Prototype لتحسين الأداء.

إضافة التوابع إلى Prototype لتحسين الأداء.

عند تصميم كائناتك باستخدام Constructor Functions، من المهم اختيار المكان الصحيح لوضع التوابع (Methods) لضمان أفضل أداء واستخدام للذاكرة.

المفهوم الأساسي: إضافة التوابع إلى Prototype

أفضل ممارسة لإضافة التوابع إلى الكائنات التي يتم إنشاؤها بواسطة Constructor Function هي إضافتها إلى خاصية `prototype` الخاصة بالـ Constructor Function نفسها. هذا يضمن أن يتم إنشاء نسخة واحدة فقط من التابع في الذاكرة، ويتم مشاركتها بواسطة جميع الكائنات التي يتم إنشاؤها من هذا Constructor. هذا يتجنب تكرار التوابع لكل كائن، مما يوفر الذاكرة ويحسن الأداء بشكل ملحوظ عند التعامل مع عدد كبير من الكائنات.

مثال برمجي

```
// الطريقة الأقل كفاءة (إنشاء تابع جديد لكل كائن)
function UserBad(name) {
  this.name = name;
  this.sayHi = function() {
    return `أهلاً، أنا ${this.name}.`;
  };
}

const userBad1 = new UserBad('خالد');
const userBad2 = new UserBad('ليلى');
console.log(userBad1.sayHi());
console.log(userBad1.sayHi === userBad2.sayHi); // false, دالتان مختلفتان

// Prototype مشاركة تابع واحد عبر الطريقة الأكثر كفاءة
function UserGood(name) {
  this.name = name;
```

```

}

UserGood.prototype.sayHi = function() {
  return `أهلاً، أنا ${this.name}.`;
};

const userGood1 = new UserGood('أحمد');
const userGood2 = new UserGood('فاطمة');
console.log(userGood1.sayHi());
console.log(userGood1.sayHi === userGood2.sayHi); // true, الدالة المشتركة

```

الشرح

في المثال الأول (`UserBad`)، يتم إنشاء دالة `sayHi` جديدة في الذاكرة لكل كائن `UserBad` يتم إنشاؤه. هذا يعني أنه إذا كان لديك 1000 كائن `UserBad`، فسيكون لديك 1000 نسخة من دالة `sayHi`، مما يهدر الذاكرة. في المقابل، في المثال الثاني (`UserGood`)، يتم إضافة دالة `sayHi` إلى `UserGood.prototype`. يتم إنشاء نسخة واحدة فقط من `sayHi`، ويتم مشاركتها بين جميع كائنات `UserGood` بفضل آلية الـ Prototype Chain. هذا يحسن استخدام الذاكرة والأداء.

- إضافة التوابع مباشرة داخل Constructor Function يؤدي إلى تكرارها لكل كائن.
- إضافة التوابع إلى `prototype` الخاص بالـ Constructor يجعلها متاحة لجميع الكائنات كنسخة واحدة مشتركة.
- هذه الممارسة تحسن بشكل كبير استهلاك الذاكرة وتزيد من كفاءة التطبيق.

Classes في ES6 — مقدمة إلى class في JavaScript

مقدمة إلى class في JavaScript

تعتبر الـ `class` في JavaScript (منذ ES6) طريقة جديدة وأكثر وضوحًا لإنشاء قوالب للكائنات (objects) والتعامل مع البرمجة كائنية التوجه (OOP). إنها تسهل تنظيم الكود الخاص بك وتجعله أكثر قابلية للقراءة والصيانة.

المفهوم الأساسي لـ class

يمكنك التفكير في الـ `class` كخريطة أو قالب لإنشاء كائنات متشابهة. على سبيل المثال، إذا كنت تريد إنشاء كائنات متعددة تمثل "السيارات"، فبدلاً من تعريف كل سيارة على حدة، يمكنك إنشاء `class` واحد باسم `Car` يحدد الخصائص (مثل اللون والماركة) والسلوكيات (مثل القيادة والتوقف) التي تشترك فيها جميع السيارات.

مثال على الكود

```
class Car {  
  // هنا سنضيف الخصائص والتوابع لاحقاً  
}  
  
// class من الـ (instance) إنشاء كائن  
const myCar = new Car();  
console.log(myCar);
```

شرح الكود

في هذا المثال البسيط، قمنا بتعريف `class` اسمه `Car` باستخدام الكلمة المفتاحية `class`.
حالياً، هذا الـ `class` فارغ ولا يحتوي على أي خصائص أو توابع. بعد ذلك، قمنا بإنشاء كائن جديد (أو

مثيل - instance) من هذا الـ `class` باستخدام الكلمة المفتاحية `new` ، وتخزينه في المتغير `myCar` .
عند طباعة `myCar` ، سترى كائنًا فارغًا من نوع `Car` .

- الـ `class` هو قالب لإنشاء الكائنات.
- يستخدم الكلمة المفتاحية `class` لتعريفه.
- يتم إنشاء كائنات من الـ `class` باستخدام الكلمة المفتاحية `new` .

Classes في constructor — ES6 داخل class.

constructor داخل class.

عند إنشاء كائن جديد من `class` ، غالبًا ما تحتاج إلى تهيئته بقيم ابتدائية معينة. هنا يأتي دور الدالة `constructor` . إنها دالة خاصة تُشغل تلقائيًا في كل مرة يتم فيها إنشاء مثيل جديد من الـ `class` .

وظيفة ودور constructor

الـ `constructor` هي دالة خاصة داخل الـ `class` تستخدم لتهيئة الكائن الجديد الذي يتم إنشاؤه. يمكنك تمرير وسائط (arguments) إلى الـ `constructor` لتحديد الخصائص الأولية للكائن. داخل الـ `constructor` ، تشير الكلمة المفتاحية `this` إلى الكائن الجديد الذي يتم إنشاؤه، مما يسمح لك بتعيين خصائص له.

مثال على الكود

```
class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }
}

const person1 = new Person('أحمد', 30);
const person2 = new Person('سارة', 25);

console.log(person1.name); // أحمد
console.log(person2.age); // 25
```

شرح الكود

في هذا المثال، قمنا بتعريف `class` اسمه `Person`. داخل هذا الـ `class`، يوجد `constructor` يقبل وسيطين: `name` و `age`. عندما ننشئ كائنًا جديدًا مثل `new Person('أحمد', 30)`، يتم استدعاء الـ `constructor` تلقائيًا، ويتم تعيين قيم `'أحمد'` و `30` لخصائص `this.name` و `this.age` للكائن الجديد. هذا يضمن أن كل كائن `Person` يتم إنشاؤه لديه اسم وعمر محددين.

- الـ `constructor` هي دالة خاصة تُنفذ عند إنشاء كائن جديد.
- تُستخدم لتهيئة الخصائص الأولية للكائن.
- تشير الكلمة المفتاحية `this` داخلها إلى الكائن الحالي.

Classes في ES6 — تعريف الخصائص والتوابع.

تعريف الخصائص والتوابع.

لتصبح الكائنات مفيدة حقًا، يجب أن تكون قادرة على تخزين البيانات (الخصائص) والقيام بأشياء معينة (التوابع أو الدوال). في الـ `class`، يمكنك تعريف هذه العناصر لتمثل سمات وسلوكيات الكائنات التي سيتم إنشاؤها.

الخصائص (Properties) والتوابع (Methods)

الخصائص: هي المتغيرات التي تحمل البيانات المتعلقة بالكائن. غالبًا ما يتم تعريفها داخل الـ `constructor` باستخدام `this.propertyName = value`. **التوابع:** هي الدوال التي تحدد السلوك الذي يمكن للكائن القيام به. يتم تعريفها مباشرة داخل الـ `class`، خارج الـ `constructor`.

مثال على الكود

```
class Dog {
  constructor(name, breed) {
    this.name = name;    // خاصية
    this.breed = breed;  // خاصية
  }

  bark() { // تابع
    return `${this.name} هاو هاو`;
  }

  describe() { // تابع آخر
    return `${this.breed}.` ومن سلالة ${this.name} هذا الكلب اسمه`;
  }
}

const myDog = new Dog('جيرمان شيبورد', 'بلاك');
console.log(myDog.bark());
console.log(myDog.describe());
```

شرح الكود

في هذا المثال، لدينا `class` اسمه `Dog` . داخل الـ `constructor` ، قمنا بتعريف خاصيتين: `name` و `breed` . هذه الخصائص ستكون فريدة لكل كائن `Dog` يتم إنشاؤه. كما قمنا بتعريف تابعين: `bark()` و `describe()` . هذه التوابع هي دوال يمكن استدعاؤها على أي كائن `Dog` . لاحظ كيف نستخدم `this.name` للوصول إلى خاصية اسم الكلب داخل التوابع. عند إنشاء `myDog` ، يمكننا استدعاء توابعه مثل `myDog.bark()` للحصول على سلوكه.

- الخصائص تخزن البيانات المتعلقة بالكائن.
- التوابع تحدد السلوكيات أو الإجراءات التي يمكن للكائن القيام بها.
- يتم الوصول إلى الخصائص والتوابع باستخدام نقطة (`.`) بعد اسم الكائن.

Static في Classes — Static Methods و ES6

.Properties

.Static Properties و Static Methods

في بعض الأحيان، قد تحتاج إلى وظائف أو بيانات مرتبطة بالـ `class` نفسه بدلاً من أن تكون مرتبطة بمثيلات فردية منه. هنا يأتي دور الخصائص (Properties) والتوابع (Methods) الثابتة (Static). هذه العناصر لا تتطلب إنشاء كائن للوصول إليها، بل تُستدعى مباشرةً على الـ `class`.

الخصائص والتوابع الثابتة (Static)

الخصائص الثابتة: هي خصائص تنتمي إلى الـ `class` نفسه، وليس إلى أي مثيل من الـ `class`.
التوابع الثابتة: هي دوال تنتمي إلى الـ `class` نفسه. غالبًا ما تُستخدم هذه التوابع لإنشاء وظائف مساعدة أو أدوات مساعدة تتعلق بالـ `class` ولكنها لا تحتاج إلى الوصول إلى بيانات محددة لمثيل معين.

مثال على الكود

```
class MathHelper {
  static PI = 3.14159; // خاصية ثابتة

  static sum(a, b) { // تابع ثابت
    return a + b;
  }

  static multiply(a, b) {
    return a * b;
  }
}

console.log(MathHelper.PI); // 3.14159
console.log(MathHelper.sum(5, 3)); // 8
console.log(MathHelper.multiply(4, 2)); // 8
```

```
// لا يمكنك الوصول إليها عبر مثيل  
// const helper = new MathHelper();  
// console.log(helper.sum(1, 2)); // خطأ!
```

شرح الكود

في هذا المثال، لدينا `class` اسمه `MathHelper`. لقد عرفنا خاصية ثابتة `PI` وتابعين ثابتين `sum` و `multiply`. لاحظ الكلمة المفتاحية `static` قبل اسم الخاصية أو التابع. للوصول إلى هذه العناصر، فإننا لا ننشئ مثيلاً من `MathHelper`. بدلاً من ذلك، نستخدم اسم الـ `class` مباشرةً: `MathHelper.PI` أو `MathHelper.sum(5, 3)`. إذا حاولت الوصول إليها من خلال مثيل تم إنشاؤه (كما هو موضح في الجزء المعلق)، فستحصل على خطأ لأنها لا تنتمي إلى المثيل.

- الخصائص والتوابع الثابتة تنتمي إلى الـ `class` نفسه.
- لا تتطلب إنشاء كائن (مثيل) للوصول إليها.
- تُستدعى مباشرة باستخدام اسم الـ `class` (مثال: `ClassName.staticMethod()`).

الوراثة (Inheritance) — الوراثة باستخدام extends.

الوراثة باستخدام extends.

الوراثة هي أحد المبادئ الأساسية في البرمجة كائنية التوجه، وتسمح لك بإنشاء فئات جديدة (فئات فرعية) بناءً على فئات موجودة (فئات رئيسية). هذا يعزز قابلية إعادة استخدام الكود ويجعل بنية التطبيق أكثر تنظيماً.

مفهوم الوراثة

تخيل أن لديك فئة عامة مثل "شخص". يمكنك بعد ذلك إنشاء فئات أكثر تحديداً مثل "طالب" أو "معلم" ترث خصائص وتوابع "شخص". في JavaScript، نستخدم الكلمة المفتاحية `extends` لتحقيق الوراثة، مما يسمح للفئة الفرعية بالوصول إلى خصائص وتوابع الفئة الرئيسية.

مثال كود

```
class Person {
  constructor(name) {
    this.name = name;
  }

  greet() {
    return `${this.name} مرحباً، أنا`;
  }
}

class Student extends Person {
  constructor(name, studentId) {
    super(name); // استدعاء مُنشئ الفئة الأب
    this.studentId = studentId;
  }

  getStudentInfo() {
    return `${this.name}، رقم الطالب: ${this.studentId}`;
  }
}
```

```
const student1 = new Student("12345", "أحمد");  
console.log(student1.greet());  
console.log(student1.getStudentInfo());
```

شرح

في هذا المثال، أنشأنا فئة `Person` كفئة أساسية. ثم أنشأنا فئة `Student` التي `extends` (ترث من) `Person`. هذا يعني أن `Student` تكتسب جميع خصائص وتوابع `Person`. داخل مُنشئ `Student`، نستخدم `super(name)` لاستدعاء مُنشئ `Person` وتمرير الاسم. يمكن لكائن من فئة `Student` الوصول إلى تابع `greet()` من فئة `Person` بالإضافة إلى تابع `getStudentInfo()` الخاص به.

- تسمح الوراثة بإنشاء فئات فرعية ترث من فئات رئيسية.
- نستخدم الكلمة المفتاحية `extends` لتعريف الوراثة في JavaScript.
- تعزز الوراثة قابلية إعادة استخدام الكود وتنظيم الفئات.

الوراثة (Inheritance) — استخدام super().

استخدام super().

عند العمل مع الوراثة، غالبًا ما تحتاج الفئة الفرعية إلى تهيئة بعض الخصائص التي يتم تعريفها في الفئة الرئيسية. هنا يأتي دور الكلمة المفتاحية `super()`، فهي تسمح لنا باستدعاء مُنشئ الفئة الأب لضمان تهيئة الخصائص الموروثة بشكل صحيح.

مفهوم super()

نُستخدم `super()` بشكل أساسي داخل مُنشئ الفئة الفرعية لاستدعاء مُنشئ الفئة الأب. هذا يضمن أن يتم تهيئة الخصائص الموروثة بشكل صحيح قبل أن تبدأ الفئة الفرعية في تهيئة خصائصها الخاصة. من الضروري جدًا، إذا كان للفئة الفرعية مُنشئ، أن تستدعي `super()` قبل استخدام الكلمة المفتاحية `this` داخل المُنشئ، وإلا فستحدث مشكلة.

مثال كود

```
class Animal {
  constructor(name) {
    this.name = name;
  }

  makeSound() {
    return "يصدر صوتًا ما";
  }
}

class Dog extends Animal {
  constructor(name, breed) {
    super(name); // استدعاء مُنشئ Animal
    this.breed = breed;
  }

  makeSound() {
    return "!الكلب ينبج: ووف ووف";
  }
}
```



```
getDogInfo() {  
  return `${this.breed} من سلالة ${this.name} هذا الكلب هو`;  
}  
  
const myDog = new Dog("جيرمان شيبرد", "باستر");  
console.log(myDog.getDogInfo());  
console.log(myDog.makeSound());
```

شرح

في هذا المثال، الفئة `Dog` ترث من `Animal`. داخل مُنشئ `Dog`، نستخدم `super(name)` لتهيئة خاصية `name` التي يتم تعريفها في الفئة الأب `Animal`. بعد استدعاء `super()`، يمكننا بأمان استخدام `this` لتعريف خصائص خاصة بالفئة `Dog` مثل `breed`. هذا يضمن أن جميع الخصائص الضرورية للفئة الأب قد تم تعيينها قبل المضي قدماً في تهيئة الفئة الفرعية.

- نستخدم `super()` لاستدعاء مُنشئ الفئة الأب من مُنشئ الفئة الفرعية.
- يجب استدعاء `super()` قبل استخدام `this` في مُنشئ الفئة الفرعية.
- تضمن `super()` تهيئة خصائص الفئة الأب بشكل صحيح وضروري.

الوراثة (Inheritance) — تجاوز التوابع (Method Overriding).

تجاوز التوابع (Method Overriding).

تجاوز التوابع هو ميزة قوية في الوراثة تسمح للفئة الفرعية بتقديم تطبيقها الخاص لتابع (دالة) موجود بالفعل في فئتها الرئيسية. هذا يعني أن التابع له نفس الاسم والبارامترات، ولكن بسلوك مختلف يناسب الفئة الفرعية.

مفهوم تجاوز التوابع

عندما تقوم فئة فرعية بتعريف تابع له نفس اسم تابع موجود في الفئة الأب، فإن التابع في الفئة الفرعية "يتجاوز" التابع في الفئة الأب. عند استدعاء هذا التابع على كائن من الفئة الفرعية، سيتم تنفيذ تطبيق الفئة الفرعية، وليس تطبيق الفئة الأب. يمكن استخدام `super.methodName()` داخل التابع المتجاوز في الفئة الفرعية لاستدعاء تطبيق الفئة الأب إذا لزم الأمر، ثم إضافة سلوك إضافي.

مثال كود

```
class Shape {
  draw() {
    return "رسم شكل عام";
  }
}

class Circle extends Shape {
  draw() {
    return "رسم دائرة." // تجاوز التابع
  }
}

class Square extends Shape {
  draw() {
    // يمكن استدعاء التابع الأصلي ثم إضافة سلوك
    const parentDraw = super.draw();
  }
}
```

```
        return `${parentDraw} ثم رسم مربع محدد  
    }  
}
```

```
const genericShape = new Shape();  
const circle = new Circle();  
const square = new Square();  
  
console.log(genericShape.draw());  
console.log(circle.draw());  
console.log(square.draw());
```

شرح

في هذا المثال، الفئة `Shape` لديها تابع `draw()`. الفئة `Circle` ترث من `Shape` وتوفر تطبيقها الخاص للتابع `draw()`، مما يؤدي إلى تجاوز التابع الأصلي. عندما نستدعي `draw()` على كائن `Circle`، يتم تنفيذ تطبيق `Circle`. أما الفئة `Square`، فهي أيضًا تتجاوز `draw()` ولكنها تستخدم `super.draw()` لاستدعاء تطبيق الفئة الأب أولاً، ثم تضيف سلوكًا خاصًا بها، مما يوضح كيفية دمج السلوك الأبوي مع السلوك الجديد.

- يسمح تجاوز التوابع للفئات الفرعية بتقديم تطبيقها الخاص لتابع موروث.
- يجب أن يكون للتابع المتجاوز نفس الاسم في الفئة الأب والفرعية.
- يمكن استخدام `super.methodName()` داخل التابع المتجاوز للوصول إلى تطبيق الفئة الأب.

التغليف والخصائص الخاصة — مفهوم Encapsulation.

مفهوم Encapsulation.

التغليف (Encapsulation) هو أحد الأعمدة الأساسية للبرمجة كائنية التوجه. يشير إلى تجميع البيانات (الخصائص) والوظائف (التوابع) التي تعمل على هذه البيانات في وحدة واحدة، وهي الفئة، وإخفاء التفاصيل الداخلية لتلك الوحدة عن العالم الخارجي.

مفهوم التغليف والخصائص الخاصة

الهدف الرئيسي من التغليف هو حماية البيانات من الوصول أو التعديل غير المصرح به، مما يحافظ على سلامة الكائن. في JavaScript، يمكننا تحقيق التغليف باستخدام الخصائص الخاصة. يتم تعريف الخصائص الخاصة باستخدام علامة # قبل اسم الخاصية، مما يجعلها غير قابلة للوصول مباشرة من خارج الفئة، ويمكن التفاعل معها فقط من خلال التوابع العامة (setters و getters).

مثال كود

```
class BankAccount {  
  #balance; // خاصية خاصة  
  
  constructor(initialBalance) {  
    if (initialBalance < 0) {  
      throw new Error("الرصيد الأولي لا يمكن أن يكون سالبًا");  
    }  
    this.#balance = initialBalance;  
  }  
  
  deposit(amount) {  
    if (amount > 0) {  
      this.#balance += amount;  
    }  
  }  
}
```

```

withdraw(amount) {
  if (amount > 0 && amount <= this.#balance) {
    this.#balance -= amount;
  } else {
    console.log("رصيد غير كافٍ أو مبلغ غير صالح للسحب");
  }
}

getBalance() {
  return this.#balance;
}
}

const myAccount = new BankAccount(100);
myAccount.deposit(50);
// console.log(myAccount.#balance); // لا يمكن الوصول لخاصية خاصة مباشرة
console.log("الرصيد الحالي:", myAccount.getBalance());
myAccount.withdraw(30);
console.log("الرصيد بعد السحب:", myAccount.getBalance());
myAccount.withdraw(200); // محاولة سحب أكثر من الرصيد

```

شرح

في هذا المثال، لدينا الفئة `BankAccount` مع خاصية خاصة `balance#`. لا يمكن الوصول إلى `balance#` أو تعديلها مباشرة من خارج الفئة. بدلاً من ذلك، نستخدم التوابع العامة مثل `deposit()` و `withdraw()` و `getBalance()` للتفاعل مع هذه الخاصية. هذا يضمن أن يتم تحديث الرصيد بطريقة متحكم بها ويمنع الأخطاء المحتملة أو التعديلات غير الصالحة، مما يحمي سلامة بيانات الكائن.

- التغليف هو تجميع البيانات والتوابع في وحدة واحدة وإخفاء التفاصيل الداخلية.
- نستخدم الخصائص الخاصة (`propertyName#`) في JavaScript لتحقيق التغليف.
- تساعد الخصائص الخاصة على حماية البيانات وتوفير واجهة متحكم بها للتفاعل مع الكائن.

Interfaces محاكاة — Abstraction و Polymorphism في JavaScript.

محاكاة Interfaces في JavaScript.

في البرمجة كائنية التوجه، تُعد الواجهات (Interfaces) طريقة لتعريف العقود التي يجب أن تلتزم بها الكائنات. بينما لا تدعم JavaScript الواجهات بشكل صريح مثل لغات أخرى، يمكننا محاكاتها لتحقيق التجريد (Abstraction) وتعدد الأشكال (Polymorphism) في تطبيقاتنا.

المفهوم الأساسي: التجريد وتعدد الأشكال

الواجهات تفرض هيكلًا سلوكيًا دون تحديد تفاصيل التنفيذ. هذا يسمح لكائنات مختلفة بامتلاك نفس الواجهة، لكن بتطبيقات خاصة بها، مما يدعم تعدد الأشكال. في JavaScript، يمكننا تحقيق ذلك من خلال التأكد من أن الكائنات تحتوي على وظائف معينة.

مثال الكود

```
class Shape {
  draw() {
    throw new Error("'draw()' يجب تنفيذ طريقة");
  }
}

class Circle extends Shape {
  draw() {
    console.log("رسم دائرة");
  }
}

class Rectangle extends Shape {
  draw() {
    console.log("رسم مستطيل");
  }
}
```

```
function renderShape(shape) {
  if (!(shape instanceof Shape) || typeof shape.draw !== 'function') {
    console.error("draw. ويمتلك طريقة Shape الكائن يجب أن يكون من نوع");
    return;
  }
  shape.draw();
}

const myCircle = new Circle();
const myRectangle = new Rectangle();

renderShape(myCircle);
renderShape(myRectangle);
```

شرح الكود

في هذا المثال، نحكي واجهة باستخدام فئة أساسية (Shape) تفرض وجود طريقة draw () عن طريق إلقاء خطأ إذا لم يتم تجاوزها. الفئات الفرعية (Circle و Rectangle) تطبق هذه الطريقة بطريقتها الخاصة. الدالة renderShape تقبل أي كائن يُحاكي واجهة Shape ويحتوي على طريقة draw ، مما يوضح مفهوم تعدد الأشكال. هذا النهج يضمن أن أي كائن يمرر إلى renderShape سيعرف كيفية الرسم.

- **التجريد:** Shape تحدد عقداً لـ draw () دون تفاصيل التنفيذ.
- **تعدد الأشكال:** renderShape تتعامل مع Circle و Rectangle بنفس الطريقة بفضل الواجهة المحاكاة.
- **التحقق اليدوي:** يتم التأكد من وجود الوظيفة المطلوبة يدوياً لضمان الالتزام بالعقد.

Design Patterns الأساسية — Factory Pattern.

Factory Pattern.

نمط المصنع (Factory Pattern) هو أحد أنماط التصميم التكوينية التي توفر واجهة لإنشاء الكائنات في فئة أساسية، ولكن تسمح للفئات الفرعية بتغيير نوع الكائنات التي سيتم إنشاؤها. يساعد هذا النمط في فصل الكود الذي ينشئ الكائنات عن الكود الذي يستخدمها.

المفهوم الأساسي: فصل إنشاء الكائنات

تخيل أن لديك تطبيقًا يحتاج إلى إنشاء أنواع مختلفة من الكائنات (مثل أنواع مختلفة من المنتجات أو المستخدمين) بناءً على بعض المعايير. بدلاً من استخدام العديد من عبارات `if/else` أو `switch` في كل مرة تحتاج فيها إلى إنشاء كائن جديد، يمكنك استخدام مصنع مركزي ليتولى هذه المهمة بذكاء ومرونة.

مثال الكود

```
class Car {
  constructor(model) {
    this.model = model;
  }
  drive() {
    return `Driving a ${this.model} Car.`;
  }
}

class Truck {
  constructor(model) {
    this.model = model;
  }
  drive() {
    return `Driving a ${this.model} Truck.`;
  }
}

class VehicleFactory {
```



```

createVehicle(type, model) {
  switch (type) {
    case 'car':
      return new Car(model);
    case 'truck':
      return new Truck(model);
    default:
      throw new Error('نوع المركبة غير مدعوم');
  }
}

const factory = new VehicleFactory();

const myCar = factory.createVehicle('car', 'Tesla Model 3');
const myTruck = factory.createVehicle('truck', 'Ford F-150');

console.log(myCar.drive());
console.log(myTruck.drive());

```

شرح الكود

في هذا المثال، لدينا فئتان `Car` و `Truck`، تقوم فئة `VehicleFactory` بدور المصنع. لديها طريقة `createVehicle` التي تأخذ نوع المركبة والموديل كمدخلات، وتقوم بإنشاء وإرجاع الكائن المناسب (إما `Car` أو `Truck`). هذا يفصل منطق إنشاء الكائن عن الكود الذي يطلب الكائنات، مما يجعل إضافة أنواع جديدة من المركبات أسهل دون تعديل الكود الحالي الذي يستخدم المصنع.

- **فصل الاهتمامات:** منطق إنشاء الكائن معزول داخل المصنع.
- **المرونة:** يمكن إضافة أنواع جديدة من الكائنات بسهولة عن طريق تعديل المصنع فقط.
- **سهولة الاستخدام:** الكود الذي يطلب الكائنات لا يحتاج إلى معرفة تفاصيل إنشاء الكائن.

Design Patterns الأساسية — Singleton Pattern.

.Singleton Pattern

نمط الفردي (Singleton Pattern) هو نمط تصميم تكويني يضمن وجود نسخة واحدة فقط من فئة معينة، ويوفر نقطة وصول عالمية (global access point) لتلك النسخة. إنه مفيد عندما تحتاج إلى كائن واحد فقط لإدارة مورد مشترك، مثل قاعدة بيانات أو مدير تهيئة.

المفهوم الأساسي: نسخة واحدة فريدة

الهدف الرئيسي لنمط Singleton هو التحكم في إنشاء الكائنات لضمان ألا يتم إنشاء أكثر من كائن واحد من فئة معينة. هذا يمنع حالات التضارب ويضمن الاتساق عند التعامل مع الموارد التي يجب أن تكون فريدة على مستوى التطبيق بأكمله.

مثال الكود

```
class DatabaseConnection {
  constructor() {
    if (DatabaseConnection.instance) {
      return DatabaseConnection.instance;
    }
    this.connection = "Connected to Database.";
    DatabaseConnection.instance = this;
    console.log("إنشاء اتصال قاعدة بيانات جديد");
  }

  getConnectionStatus() {
    return this.connection;
  }
}

const db1 = new DatabaseConnection();
console.log(db1.getConnectionStatus());

const db2 = new DatabaseConnection();
console.log(db2.getConnectionStatus());
```

```
console.log(db1 === db2); // يجب أن تكون true
```

شرح الكود

في هذا المثال، لدينا فئة `DatabaseConnection` . في دالة البناء (constructor)، نتحقق مما إذا كانت هناك نسخة موجودة بالفعل (`DatabaseConnection.instance`). إذا كانت موجودة، فإننا نُعيد تلك النسخة بدلاً من إنشاء واحدة جديدة. إذا لم تكن موجودة، نقوم بإنشاء النسخة وتخزينها في `DatabaseConnection.instance` قبل إرجاعها. هذا يضمن أن كلا من `db1` و `db2` يشيران إلى نفس الكائن، مما يؤكد مبدأ Singleton.

- **نسخة فريدة:** يضمن وجود نسخة واحدة فقط من الفئة.
- **نقطة وصول عالمية:** يوفر طريقة موحدة للوصول إلى هذه النسخة.
- **إدارة الموارد:** مفيد لإدارة الموارد المشتركة مثل اتصالات قاعدة البيانات.

Design Patterns الأساسية — Observer Pattern.

Observer Pattern.

نمط المراقب (Observer Pattern) هو نمط تصميم سلوكي يُستخدم لتحديد علاقة واحد إلى متعدد بين الكائنات، حيث عندما يتغير كائن واحد (الموضوع - Subject)، يتم إعلام جميع الكائنات التابعة له (المراقبون - Observers) تلقائيًا وتحديثها. هذا النمط يعزز الاقتران الضعيف (loose coupling) بين المكونات.

المفهوم الأساسي: إعلام الكائنات التابعة

تخيل نظامًا حيث يحتاج العديد من الأجزاء المختلفة في تطبيقك إلى التفاعل أو التحديث عند حدوث تغيير معين (حدث). بدلاً من أن تقوم الأجزاء الأخرى بالتحقق باستمرار من التغييرات، يقوم نمط المراقب بإنشاء آلية اشتراك، حيث "تستمع" المكونات المهتمة للحدث، وعندما يحدث، يتم إعلامها تلقائيًا.

مثال الكود

```
class Subject {
  constructor() {
    this.observers = [];
  }

  addObserver(observer) {
    this.observers.push(observer);
  }

  removeObserver(observer) {
    this.observers = this.observers.filter(obs => obs !== observer);
  }

  notify(data) {
    this.observers.forEach(observer => observer.update(data));
  }
}

class Observer {
```

```

constructor(name) {
  this.name = name;
}

update(data) {
  console.log(` ${this.name} تلقى تحديثًا: ${data}`);
}
}

const newsPublisher = new Subject();

const subscriber1 = new Observer("المشترك 1");
const subscriber2 = new Observer("المشترك 2");
const subscriber3 = new Observer("المشترك 3");

newsPublisher.addObserver(subscriber1);
newsPublisher.addObserver(subscriber2);
newsPublisher.addObserver(subscriber3);

newsPublisher.notify("اكتشاف كوكب جديد");

newsPublisher.removeObserver(subscriber2);

newsPublisher.notify("تحديث: مؤتمر صحفي بعد ساعة");

```

شرح الكود

في هذا المثال، لدينا `Subject` (الموضوع، وهو ناشر الأخبار) الذي يمكن للمراقبين (`Observer`، وهم المشتركون) التسجيل فيه أو إلغاء التسجيل منه. عندما يحدث شيء مهم (مثل نشر خبر جديد)، يقوم `Subject` باستدعاء طريقة `notify`، والتي بدورها تقوم باستدعاء طريقة `update` لكل مراقب مسجل. بهذه الطريقة، يتم إعلام جميع المشتركين تلقائيًا بالحدث دون الحاجة إلى معرفة تفاصيل بعضهم البعض، مما يوضح مبدأ الاقتران الضعيف.

- **الاقتران الضعيف:** الموضوع والمراقبون لا يعرفون تفاصيل بعضهم البعض.
- **قابلية التوسع:** يمكن إضافة مراقبين جدد بسهولة دون تعديل الموضوع.
- **الإعلام التلقائي:** يتم إعلام المراقبين تلقائيًا عند تغيير حالة الموضوع.